

高级语言程序设计

实验报告

南开大学 计算机大类

姓名：李树青

学号：2512885

班级：计算机伯苓班

2026 年 5 月 12 日

一、作业题目

《罗小黑向上冲》（Luo Xiao Hei Jump Adventure）

二、项目简介

本项目使用 C++ 开发一款 2D 纵向跳跃小游戏。玩家控制罗小黑左右移动，通过自动跳跃踩踏木板持续向上攀登，同时需要躲避悬空危险机关，最终到达顶部通关。游戏小黑风格的 UI 与角色动作，代码以面向对象方式拆分为总控、角色、平台三大模块，最终可打包为 exe + assets/ 目录结构，便于分发运行。

三、开发环境

本项目基于以下环境开发：开发语言为 C++（C++20 标准）；开发工具为 Visual Studio 2026（MSVC v145）；图形库使用 EasyX（graphics.h）；音频接口使用 Windows MCI（mciSendString）；运行平台为 Windows。

四、主要功能

4.1 基础玩法

玩家通过键盘左右方向键控制角色水平移动，角色下落时踩到木板会自动起跳。从屏幕左侧出界后会从右侧出现，反之亦然，形成屏幕穿越效果。若角色掉出屏幕底部则触发失败界面，到达顶端高度则触发通关界面。

4.2 木板与机关

场景中包含三类对象：普通木板作为基础落脚点；蓝玉盘（跳跃板）踩后触发超级跳跃，使角色弹射至更高处；悬空危险机关与玩家碰撞后直接触发失败。

4.3 界面与交互

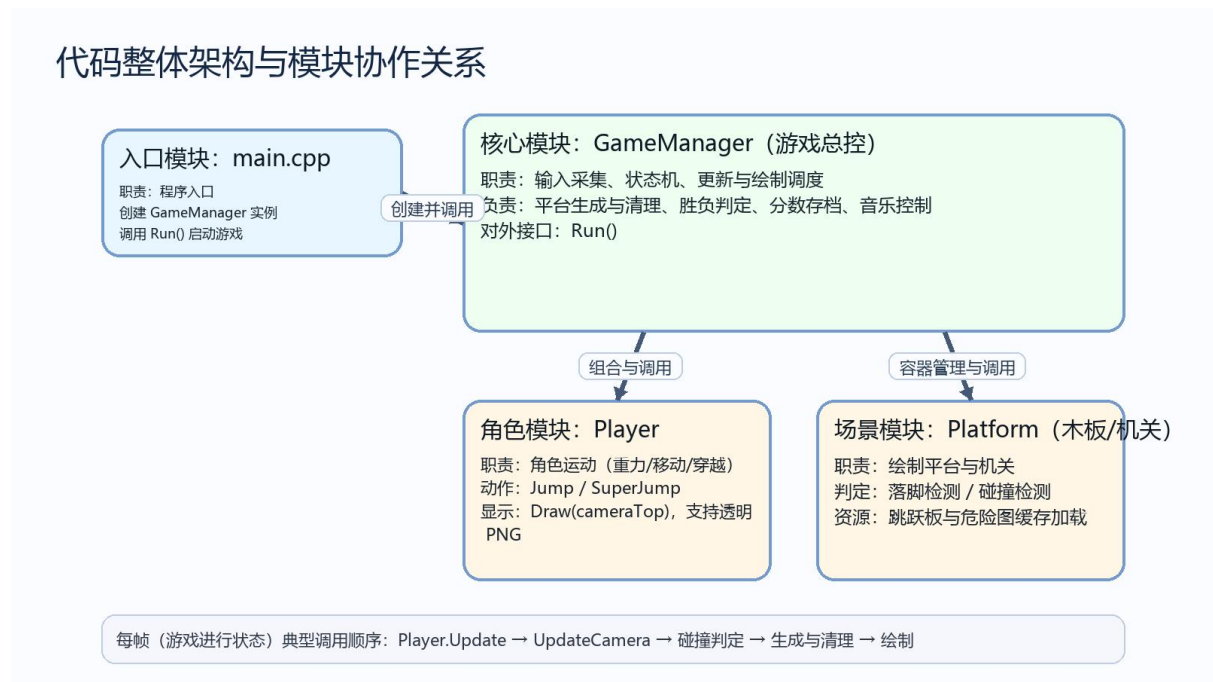
游戏包含完整的界面流转：开始菜单、游戏进行、继续游戏、失败界面以及通关界面。

五、主要流程

5.1 整体代码框架

整体代码按“入口→总控调度→角色/平台模块”的思路组织。

程序运行的主流程为：初始化窗口 → 进入主循环 → 每帧执行输入/更新/绘制 → 退出时清理资源。



5.2 分模块讲解

5.2.1 入口模块: main.cpp

程序入口，负责创建 GameManager 实例并调用 Run() 启动游戏。

5.2.2 总控模块: GameManager.h / GameManager.cpp

总控模块是整个游戏的调度核心，主要包含以下功能：

Run() 实现主循环（输入→更新→绘制），并依据 state 切换开始/暂停/失败/胜利界面；ResetGame() 负责开局初始化，包括玩家位置、相机、平台容器、背景缓存与音乐播放；UpdateCamera() 更新 cameraTop，实现画面向上推进与背景滚动；CheckCollisions() 统一处理碰撞规则——踩木板触发跳跃，碰危险机关触发失败；GeneratePlatforms() 与 CleanPlatforms() 动态生成新木板/机关，并删除屏幕下方的旧对象；DrawBackground() 及各 DrawXXX() 函数分别负责背景与不同状态下的 UI

绘制。

5.2.3 角色模块: Player.h / Player.cpp

角色模块封装了玩家的所有行为。Update(leftKey, rightKey) 处理左右移动、屏幕穿越及重力下落; Jump() 与 SuperJump() 在总控判定踩板成立后被调用, 分别改变普通/超级竖直速度; Draw(cameraTop) 则根据角色当前状态绘制对应贴图。

5.2.4 平台与机关模块: Platform.h / Platform.cpp

平台模块负责所有场景对象的绘制与判定。Draw(cameraTop) 绘制普通木板、跳跃板标识及危险机关 PNG; CheckLanding(...) 执行落脚判定, 命中后由总控识别板类型并决定调用 Jump 还是 SuperJump; CheckCollision(...) 执行危险机关碰撞判定, 命中后触发失败逻辑; LoadDeathImage() 与 LoadJumpImage() 加载并缓存 PNG 资源, 避免每帧重复加载导致卡顿。

5.3 关键函数代码讲解

5.3.1 透明 PNG 绘制 (角色/按钮/机关通用)

该函数解决了 EasyX 直接绘制 PNG 时容易出现黑边的问题, 是视觉效果最关键的实现。通过直接按 PNG 的 Alpha 通道进行混合, 角色与按钮边缘更干净, 画面更精细。

```
// Player.cpp (节选)
inline void putimage_alpha(int x, int y, const IMAGE* img)
{
    int w = img->getwidth();
    int h = img->getheight();
    AlphaBlend(GetImageHDC(NULL), x, y, w, h,
               GetImageHDC(img), 0, 0, w, h,
               { AC_SRC_OVER, 0, 255, AC_SRC_ALPHA });
}
```

5.3.2 Run(): 主循环

Run() 是整个游戏的总循环, 统一管理“输入→更新→绘制”的节奏, 保证不同状态下的界面与逻辑切换稳定流畅。

```
// GameManager.cpp (节选)
void GameManager::Run() {
```

```

initgraph(SCREEN_WIDTH, SCREEN_HEIGHT);
SetWindowText(GetHWnd(), L"Luo Xiao Hei Jump Adventure");
BeginBatchDraw();
ResetGame();
state = START_MENU;
while (true) {
    if (!IsWindow(GetHWnd())) break;
    bool leftKey = (GetAsyncKeyState(VK_LEFT) & 0x8000) != 0;
    bool rightKey = (GetAsyncKeyState(VK_RIGHT) & 0x8000) != 0;
    // 处理鼠标点击: 开始/暂停/继续/重开
    if (state == PLAYING) {
        player.Update(leftKey, rightKey);
        UpdateCamera();
        CheckCollisions();
        GeneratePlatforms();
        CleanPlatforms();
        CheckGameOver();
        CheckWin();
    }
    cleardevice();
    DrawBackground();
    // 按 state 绘制菜单/游戏/暂停/失败/胜利界面
    FlushBatchDraw();
    Sleep(16);
}
EndBatchDraw();
closegraph();
}

```

5.3.3 落脚判定 CheckLanding()

落脚判定决定了自动起跳的触发条件，是玩法成立的核心。只有满足下落状态+位置穿过木板顶部+横向落在板上三个条件，才算有效落脚，从而避免上升时意外黏板。

```

// Platform.h (节选)
bool CheckLanding(int playerX, int playerY, int playerRadius, int
playerVy) const {
    if (playerVy <= 0) return false;
    int footY = playerY + playerRadius;
    int prevFootY = footY - playerVy;
    int left = playerX - playerRadius;
    int right = playerX + playerRadius;
    if (footY >= y && prevFootY <= y + 8) {
        if (right > x + 5 && left < x + width - 5) {
            return true;
        }
    }
}

```

```
return false;
}
```

六、实现效果



图 1 开始菜单界面

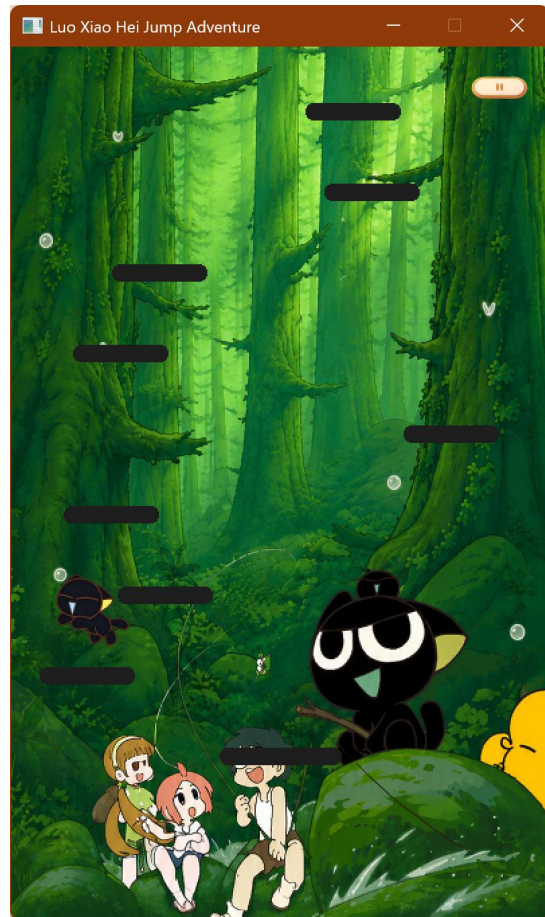


图 2 游戏进行界面

程序运行稳定，界面切换正常，角色与按钮 PNG 透明效果正常，关卡可持续生成，玩法完整。

七、使用 AI 披露

7.1 使用工具

本项目开发过程中使用了以下 AI 工具：Claude 3.5 Sonnet 用于辅助代码开发，主演用来提出结构建议与实现思路，生成部分片段；豆包用于辅助修改代码，帮助我定位问题，提出改动建议；ChatGPT Image 2 用于辅助图片素材处理，主要负责背景拼接与图片融合。

7.2 使用位置与示例

示例 1：透明 PNG 绘制思路（参考 Claude 建议并落地实现）

```
// Player.cpp (节选)
inline void putimage_alpha(int x, int y, const IMAGE* img)
{
    int w = img->getwidth();
    int h = img->getheight();
    AlphaBlend(GetImageHDC(NULL), x, y, w, h,
               GetImageHDC(img), 0, 0, w, h,
               { AC_SRC_OVER, 0, 255, AC_SRC_ALPHA });
}
```

示例 2：背景音乐路径调整（豆包辅助定位绝对路径问题，改为相对路径）

```
// GameManager.cpp (节选)
mciSendString(L"open \"assets\\bgm\\hx.mp3\" alias bgm", NULL, 0, NULL);
mciSendString(L"play bgm repeat", NULL, 0, NULL);
```

示例 3：落脚/机关判定代码检查（豆包辅助检查边界情况，本人核对并本机验证）

```
// Platform.h (节选)
bool CheckLanding(int playerX, int playerY, int playerRadius, int
playerVy) const {
    if (playerVy <= 0) return false;
    // ... (完整逻辑见 5.3.3)
    return true;
}
```

图片素材说明：UI/背景等图片由本人整理素材与筛选，部分通过 ChatGPT Image 生成/处理后，再经人工挑选并调整尺寸。如下图，原图为网上搜索，字体为 gpt 添加。



图 3 通关页面

7.3 人工核对说明

AI 输出的所有内容均由本人阅读理解、人工修改与合并，以本地编译运行结果为最终依据。对关键路径进行了人工检查与运行验证，确保代码逻辑正确、可在本机稳定运行。

八、收获与总结

8.1 面向对象知识点的学以致用

通过本次项目，我深度理解了面向对象的核心思想，将代码按职责拆分微模块 GameManager、Player、Platform，分别管理总控、角色与场景对象；我还将数据与行为封装到类中，如 Player.Update/Draw、Platform.CheckLanding/Draw；理解了组合关系的实际应用——GameManager 以成员与容器持有 Player 与多个 Platform。

8.2 项目开发中的坑与解决方案

开发过程中遇到并解决了以下几个问题：

- 1、游戏里角色、按钮、机关很多都是带透明通道的 PNG，直接用 EasyX 的 putimage 绘制时，PNG 边缘出现黑边/锯齿感明显，透明区域被当成黑色背景贴到画面上，导致 UI 和角色“糊一圈黑”，整体观感很廉价。我没有用“黑白掩码两次贴图”的老办

法，而是直接走 Windows 图形接口。用 AlphaBlend 读取 PNG 的 Alpha 通道进行混合，并封装成 `putimage_alpha`，随后在项目里统一使用它来绘制。

2、开始菜单、暂停界面、失败界面这些地方，如果每一帧都 `loadimage` 或频繁做资源加载，会造成瞬间卡顿掉帧、画面闪烁或响应变慢、CPU 占用上升。所以，我对可复用资源进行缓存，背景图只在首次进入游戏时加载并缓存，跳跃板图、危险图等通过静态成员/标志位只加载一次。从而保持主循环里每帧主要做输入读取、位置更新、碰撞判定、绘制，不做重 IO。